Informe TFM

Luis Requeni Zamora

01/01/2023

INTRODUCCIÓN

A raíz del último informe se trabaja en el mismo documento, depurando los distintos modelos planteados anteriormente. Los adaptamos al lenguaje utilizado en asignaturas del Master a traves de *jags* y evitamos así utilizar la función *jags.model* cuya salida era más complicada de interpretar. Seguimos el mismo esquema.

Para ello, hemos tratado de plantear para la matriz de componentes principales estándar asociada al dataset *melanoma* un modelo de mixtura que permita realizar una correcta clasificación de nuestros spots asociados al data.frame.

```
melanoma <- getRDS(dataset = "2018_thrane_melanoma", sample = "ST_mel1_rep2")
melanoma_int_colData <- melanoma@int_colData # Informacion PCA
melanoma_int_colData_reducedDims <- as.data.frame(melanoma_int_colData@listData$reducedDims)
```

```
datos <- melanoma_int_colData_reducedDims
head(datos)
```

```
##      PCA.PC1    PCA.PC2    PCA.PC3    PCA.PC4    PCA.PC5    PCA.PC6    PCA.PC7
## 1 -1.9829379  1.4512358  -7.227435  2.1068583 -2.1042511  0.3544527 -1.5864687
## 2 -1.5391045  0.5299336   1.203405  2.0464473 -1.8843623 -4.0249423 -0.4114954
## 3  0.2316224  1.0587171   3.356286  0.5383973  1.5228345 -0.6354552 -3.0508375
## 4 -4.0109392  2.2564052   2.351316 -1.2636699  0.5921051  0.1325020 -2.3654549
## 5  1.1661865  0.1063665  -2.860336  2.5964039 -6.1877102  1.1612939 -3.3465968
## 6  1.8583911 -1.7624881 -11.021961  5.6702354 -1.4486031 -1.4965118 -1.2718448
```

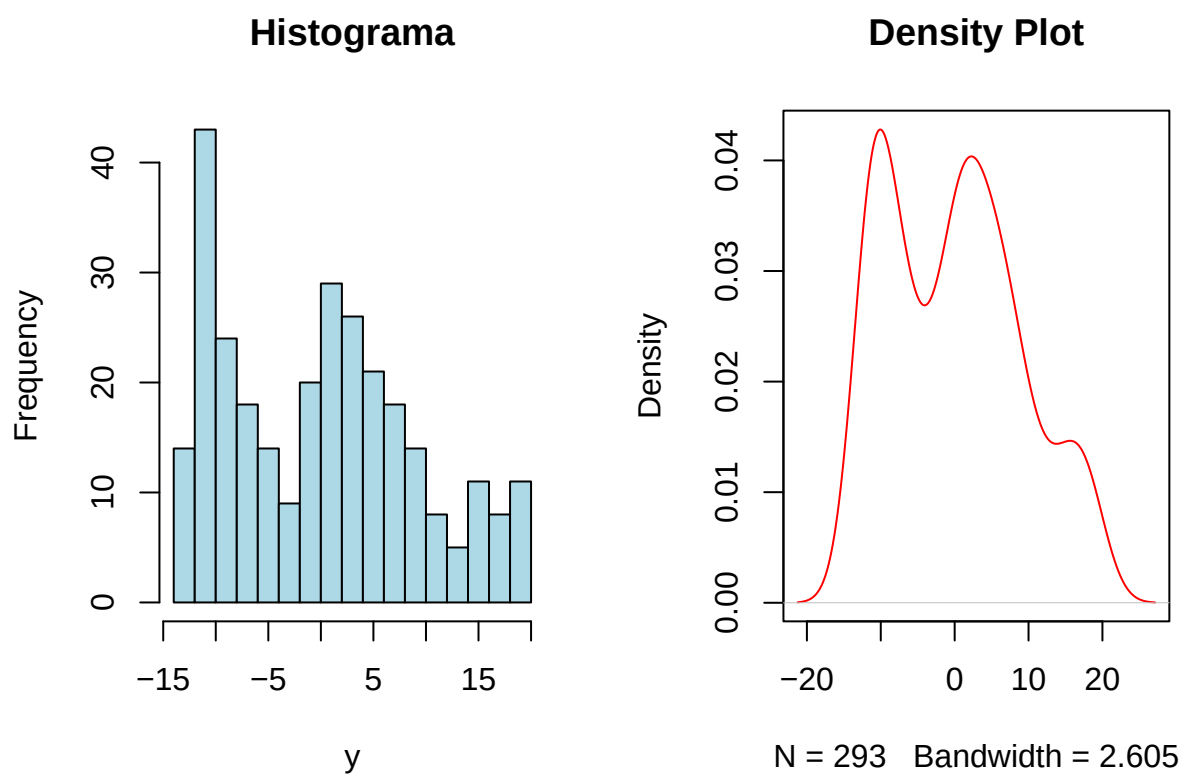
Contamos así con un data.frame con 293 observaciones (nuestros 293 spots) y 7 variables. Aunque partiremos directamente con estos datos para la construcción de nuestro modelo, más adelante realizaremos un estudio de como se ha llegado a esa matriz de PCA, tratando de valorar tanto el número de columnas necesarias para explicar una determinada cantidad de información como poder valorar cuales son los genes con una mayor variabilidad y que por tanto resultan más relevantes en esta construcción.

CONSTRUCCIÓN DEL MODELO BÁSICO

Inicialmente planteamos un modelo de mixturas únicamente para la primera componente principal para hacernos una ligera idea del objetivo que buscamos. Mostramos la construcción y explicamos los aspectos más relevantes.

- La visualización del *density.plot* de la primera componente principal podría llevarnos a pensar en dos o tres clusterings distintos.

```r
y <- datos$PCA.PC1
```



Podríamos apostar a que un modelo de mixturas con dos grupos podría representar bien esta situación. Para el primer grupo cabría esperar una media cercana a -10. En el segundo grupo esta debería girar en torno a 5. Más adelante podríamos llegar a valorar la simulación del número de grupos necesarios. Comentar, por último, que parece que debería haber más elementos en el segundo grupo que en el primero.

- A tenor de esto, planteamos el siguiente modelo:

```r
Modelo_Ejercicio <- function()
{
  for(i in 1:293)
  {
    # Para cada simulacion habra dos posibles valores de mu[i]
    # El que plante mus[1] y el propuesto por mus[2]
    # Con esta modificacion podemos ver la clasificacion para cada y[i]
    # En el informe anterior solo veiamos los valores de mus[1] y mus[2]
    y[i] ~ dnorm(mu[i], prec)
    mu[i] <- mus[z[i]]
    z[i] ~ dcat(omega)
  }
  for(k in 1:2)
  {
    mus[k] ~ dnorm(0, 1/100)
  }
```

```
  # Parece que esta distribucion inicial de gamma es adecuada
  # Investigar mas porque nos ha generado muchos problemas
  prec ~ dgamma(0.01, 0.01)
  sig = sqrt(1/prec)
  omega ~ ddirich(c(1, 1))
}
Datos_Ejercicio <- list(y = y)
```

```
Iniciales_Ejercicio <- function()
{
  list(prec = 1)
}
Parametros_Ejercicio <- c("mu", "omega", "mus", "z", "sig")
```

Asumimos que cada y_i, $i = 1, \ldots, 293$ (el valor registrado para cada uno de nuestros 293 spots en nuestra primera componente principal) sigue una distribución Normal con media $mu[i]$. Tendremos una media $mus[1]$ para el primer *cluster* y una media $mus[2]$ para el segundo.

En cuanto a la clasificación de los clusters, utilizamos la distribución categórica *dcat()* donde la probabilidad de pertenencia a cada grupo viene determinada por el parámetro *omega*. A este parámetro *omega* le hemos asociado una función de distribución Dirichlet, que para el caso particular con dos categorías es una distribución Beta. De la representación gráfica previa podríamos asumir que existe un mayor número de registros en la segunda categoría que en la primera. No obstante, para no ser informativos optamos por *ddirich(c(1, 1))*.

RESULTADOS DEL MÓDELO

```
set.seed(1)
Resul_Ejercicio <- jags(data = Datos_Ejercicio,
                        inits = Iniciales_Ejercicio,
                        parameters.to.save = Parametros_Ejercicio,
                        model.file = Modelo_Ejercicio,
                        working.directory = NULL, n.iter = 100000, n.burnin = 10000, n.thin = 10)
```

En este sentido, podemos comprobar si la construcción del modelo ha sido la adecuada, las distintas clasificaciones que se proponen para nuestros spots, así como las características de nuestros dos clusters propuestos.

- Resumen del modelo:

```
Resultados <- Resul_Ejercicio$BUGSoutput
head(Resultados$summary)
```

```
##                  mean        sd        2.5%         25%         50%         75%
## deviance 1869.2726620 33.777523 1815.395469 1845.753808 1865.512231 1888.392487
## mu[1]      -3.2625821  5.354797   -7.411940   -6.304360   -5.575528   -4.460655
## mu[2]      -2.8574784  5.637046   -7.391054   -6.257684   -5.507236   -4.130215
## mu[3]      -1.0078903  6.490014   -7.260268   -6.046823   -5.079226    7.241075
## mu[4]      -4.5596533  4.061246   -7.500854   -6.414671   -5.763057   -4.984209
## mu[5]       0.3190394  6.752558   -7.169065   -5.834457   -4.492442    7.618413
```

```
##                97.5%     Rhat n.eff
## deviance 1944.112752 1.001195  7900
## mu[1]       8.622033 1.000971 27000
## mu[2]       8.707030 1.001297  5700
## mu[3]       9.022690 1.001084 14000
## mu[4]       8.125473 1.000963 27000
## mu[5]       9.182566 1.001272  6100
```

```r
summary((Resultados$summary[, 8])) # Informacion R-hat
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.001   1.001   1.001   1.001   1.001   1.015
```

```r
summary((Resultados$summary[, 9])) # Informacion simulaciones efectivas
```

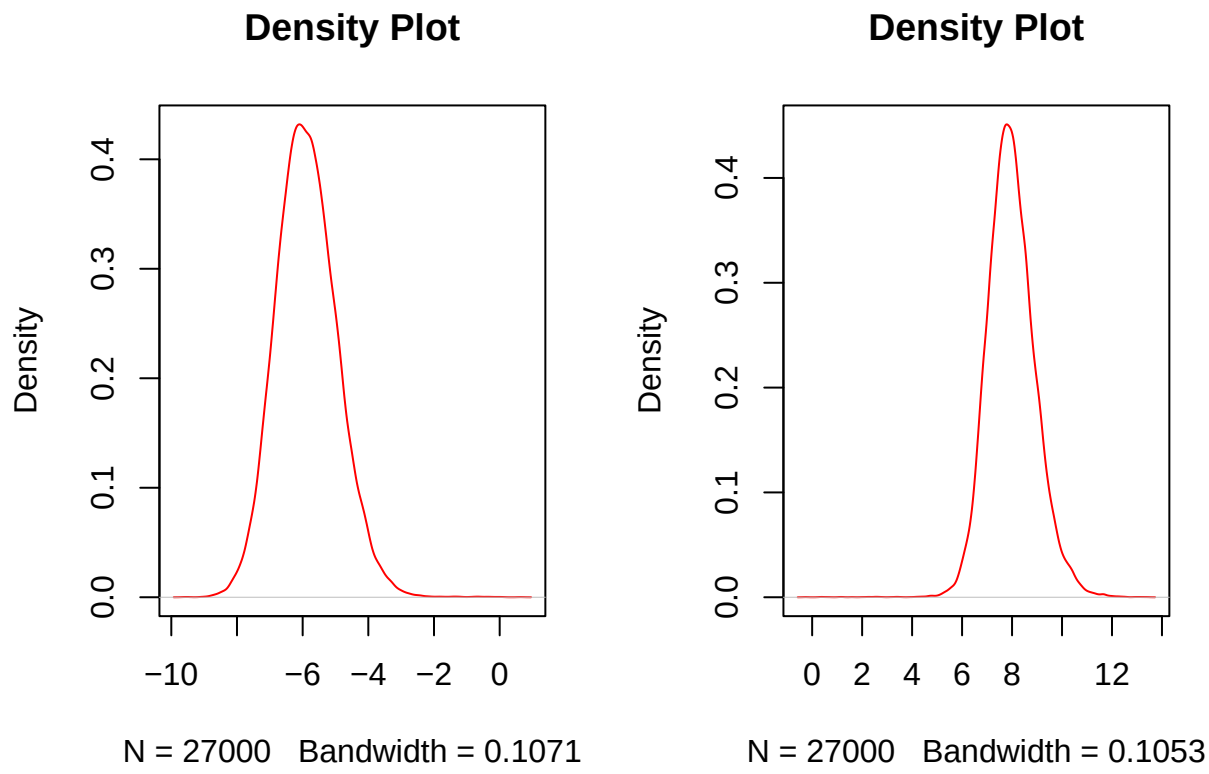
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    4600   15000   27000   21320   27000   27000
```

La construcción del modelo tanto en relación al número efectivo de simulaciones como con el *R-hat* es adecuada.

- Clasificación

Por un lado estamos interesados en conocer cual es la media para cada cluster, pero igualmente queremos conocer cuantas observaciones pertenecen a cada cluster así como valorar la clasificación que se ha realizado para alguna observación concreta.

```r
# Informacion mus
par(mfrow = c(1, 2))
plot(density(Resultados$sims.list$mus[, 1]), main = "Density Plot", col = "red")
plot(density(Resultados$sims.list$mus[, 2]), main = "Density Plot", col = "red")
```



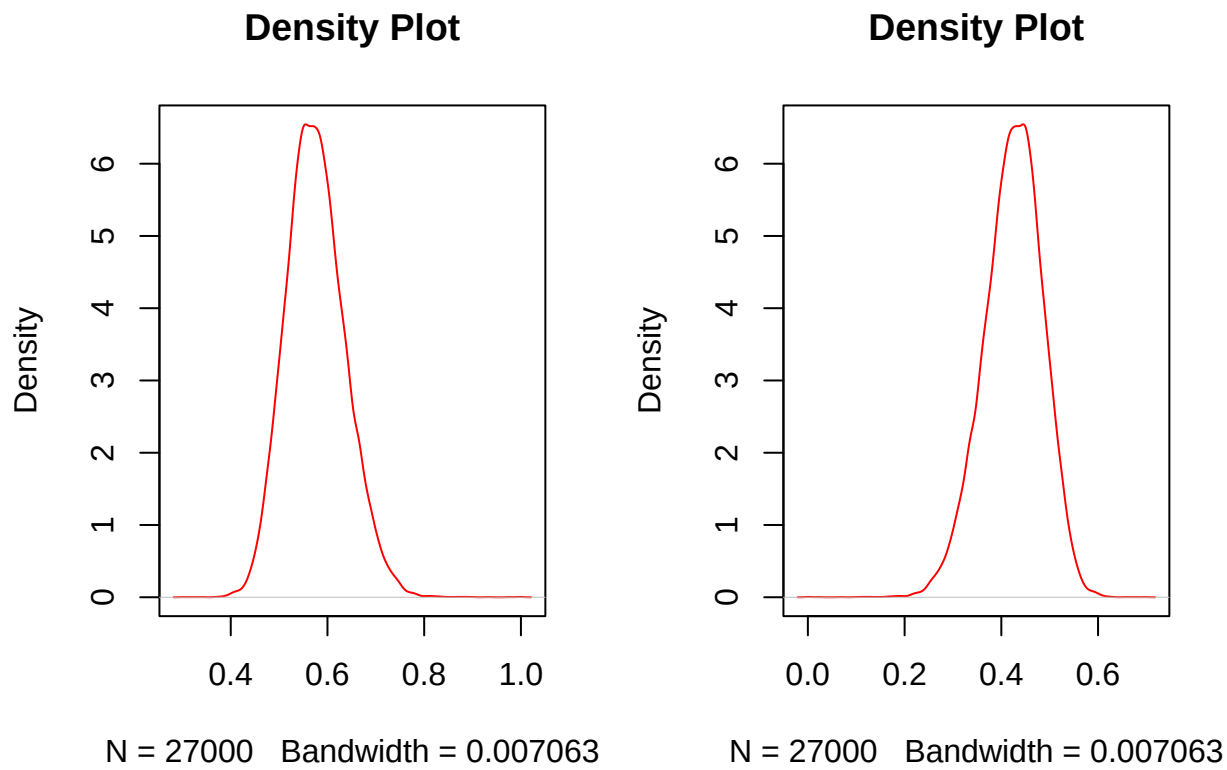

```r
summary(Resultados$sims.list$mus[, 1])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -9.6047 -6.5069 -5.9136 -5.8703 -5.2796  0.6343
```

```r
summary(Resultados$sims.list$mus[, 2])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -0.261   7.347   7.923   7.976   8.553  13.408
```

```r
# Informacion omega
par(mfrow = c(1, 2))
plot(density(Resultados$sims.list$omega[, 1]), main = "Density Plot", col = "red")
plot(density(Resultados$sims.list$omega[, 2]), main = "Density Plot", col = "red")
```




```
summary(Resultados$sims.list$omega[, 1])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.3033  0.5338  0.5728  0.5759  0.6147  0.9998
```

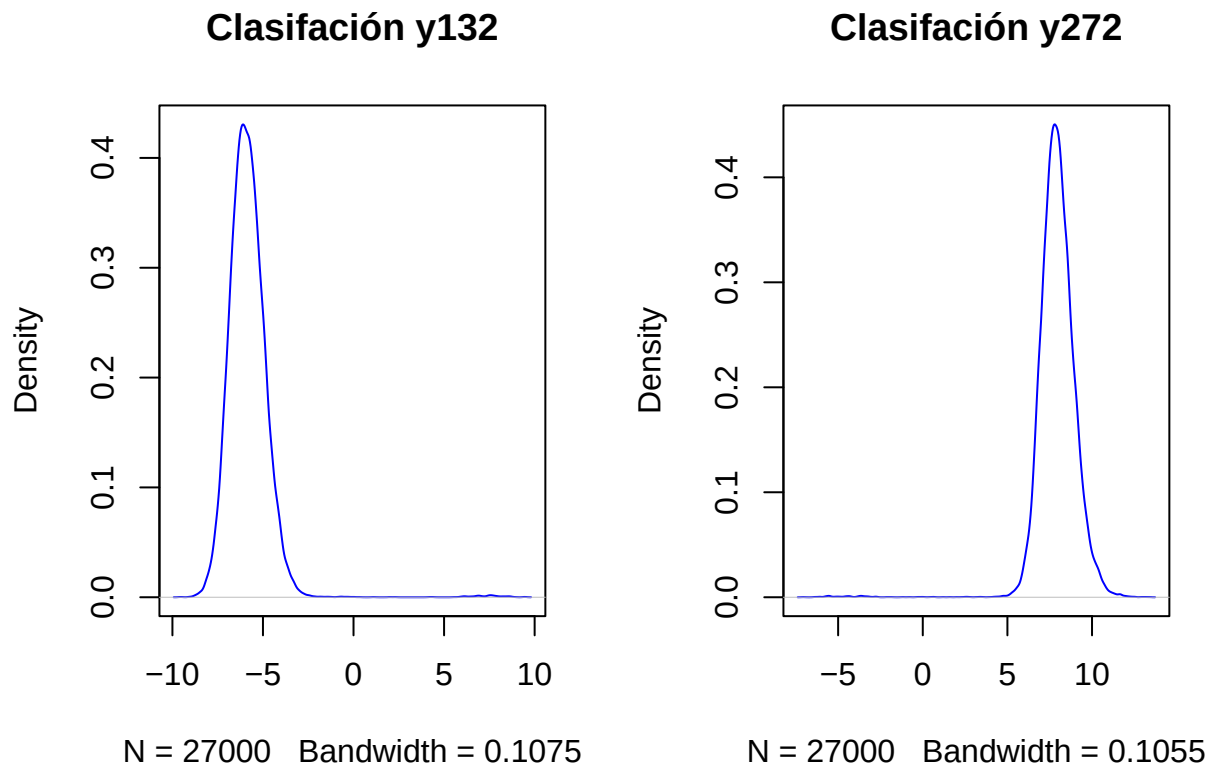
```
summary(Resultados$sims.list$omega[, 2])
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.0002148 0.3852603 0.4272448 0.4240681 0.4661931 0.6966990
```

Nos llama la atención que nuestro modelo apunta a que existen más elementos en el primer cluster que en el segundo, cuando viendo las representaciones iniciales quizá hubieramos apostado por lo contrario. Las medias para los clusters 1 y 2 se aproximan bastante a lo que cabría esperar pero parece que existe cierto margen de mejora.

Para valorar la clasificación de nuestros distintos elementos y_i, hemos cogido 5 elementos. Dos de ellos claramente pertenecientes a los grupos 1 y 2 respectivamente (y_{132} y y_{272}). Y luego tres elementos que generan más dudas (y_{29} ligeramente orientado hacia el cluster 1, y_{45} totalmente arbitrario y y_{98} más próximo al cluster 2)

```
par(mfrow = c(1, 2))
plot(density(Resultados$sims.list$mu[, 132]), main = "Clasifación y132", col = "blue")
plot(density(Resultados$sims.list$mu[, 272]), main = "Clasifación y272", col = "blue")
```



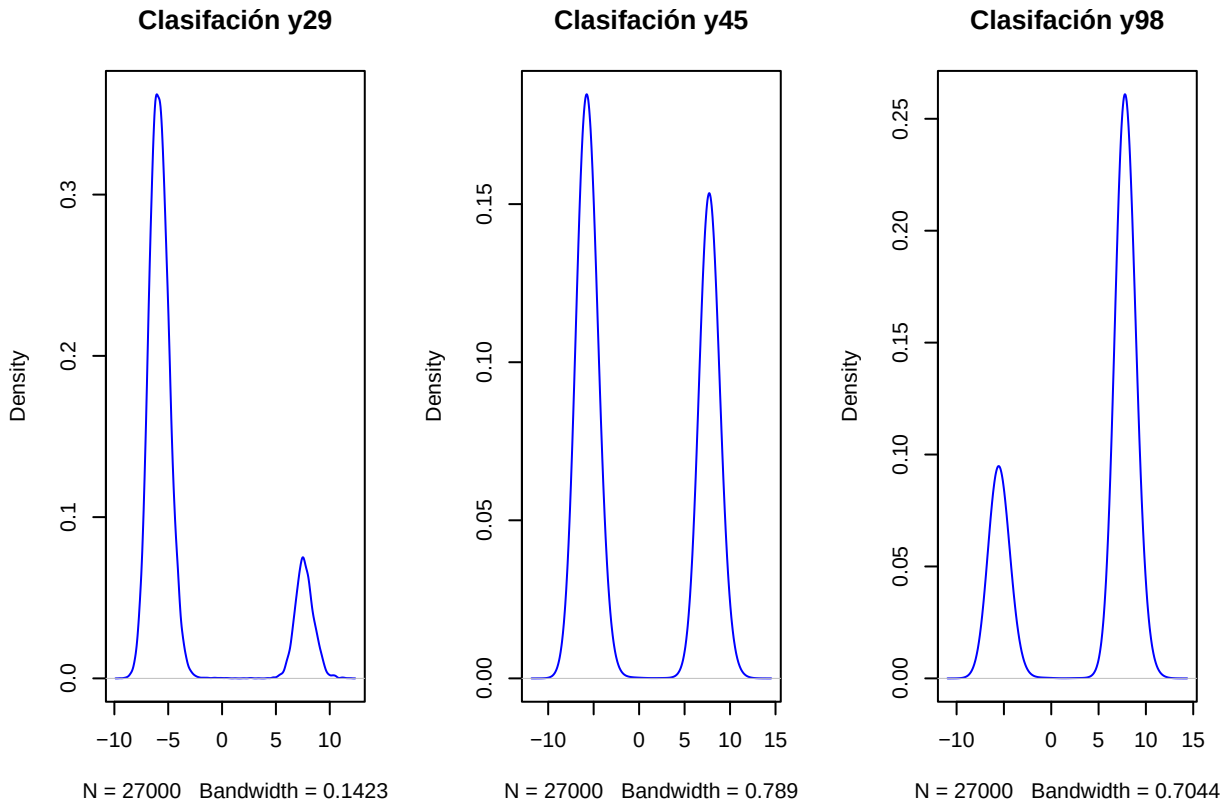

```
summary(Resultados$sims.list$mu[, 132])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -9.605  -6.504  -5.910  -5.828  -5.272   9.481
```

```
summary(Resultados$sims.list$mu[, 272])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -7.071   7.341   7.920   7.942   8.549  13.408
```

```
par(mfrow = c(1, 3))
plot(density(Resultados$sims.list$mu[, 29]), main = "Clasifación y29", col = "blue")
plot(density(Resultados$sims.list$mu[, 45]), main = "Clasifación y45", col = "blue")
plot(density(Resultados$sims.list$mu[, 98]), main = "Clasifación y98", col = "blue")
```




```
summary(Resultados$sims.list$mu[, 29])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -9.469  -6.350  -5.660  -3.753  -4.719  11.934
```

```
summary(Resultados$sims.list$mu[, 45])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -9.4687 -5.8561 -4.5222  0.2712  7.5952 12.1558
```

```
summary(Resultados$sims.list$mu[, 98])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -8.848  -4.197   7.423   4.213   8.171  12.249
```

Todas las representaciones parecen ir en consonancia con lo comentado.

Por último tratamos de solucionar los distintos problemas de *label.switching*. No parece que se produzcan en este modelo vistas las distintas representaciones gráficas, pero es posible que cuando aumente la dimensionalidad, y sea más difícil de visualizar la correcta o no clasificación, sí que se produzcan. Hacemos uso para ello de la función *label.switching()* del paquete que lleva su mismo nombre.

El problema de *label.swiching* surge como consecuencia de la inferencia a posteriori en el modelo de mixtura que imposibilita evaluar significativamente las estadísticas a posteriori utilizando los procedimientos estándar de Monte Carlo. Este problema surge como consecuencia de la invariancia a posteriori bajo las acciones de un determinado grupo. Por ejemplo, permutar el orden de los componentes de la mixtura no tiene efecto sobre la probabilidad.

Entre los distintos algoritmos que ofrece el paquete utilizamos la segunda versión iterativa de la Equivalencia de Clases Representantivas (*erc.iterative.2()*) propuesta por Papastamoulis y Iliopoulos (2010). Este modelo presecisa de un array de dimensión $m \times n \times k$ de probrabilidades de asignación para cada uno de las 293 localizaciones (n) en cada una de nuestras 27.000 iteraciones (m) y para cada uno de los 2 clusters propuestos (k).

Dado que esa información no es proporcionada por la propia MCMC, debemos construirla nosotros.

Para una determinada iteración $t = 1, \ldots, 27000$ de MCMC, sea $w_k^{(t)}$ y $\zeta_k^{(t)}$, $k = 1, 2$, los pesos simulados para cada una de las mixturas y los parámetros de las componentes específicas respectivamente. De esta manera, el elemento (t, i, k) para p se corresponde con la probabilidad condicionada de que la observación $i = 1, \ldots 293$ pertenezca a la mixtura k.

$$p_{itk} = \frac{w_k^{(t)} f(x_i|\zeta_k^{(t)})}{w_1^{(t)} f(x_i|\zeta_1^{(t)}) + w_2^{(t)} f(x_i|\zeta_2^{(t)})}$$

```
m <- Resultados$n.sims
n <- nrow(datos)
K <- 2
z <- Resultados$sims.list$z
weights <- Resultados$sims.list$omega
zheta <- Resultados$sims.list$mus
sigma <- Resultados$sims.list$sig

p <- array(dim = c(m, n, K))
for(i in 1:m){
  print(i)
  for(j in 1:n){
    for(k in 1:K){
      p[i, j, k] <- (weights[i, k]*dnorm(y[j], mean = zheta[i, k], sd = sigma[i])) /
                    ((weights[i, 1]*dnorm(y[j], mean = zheta[i, 1], sd = sigma[i])) +
                     (weights[i, 2]*dnorm(y[j], mean = zheta[i, 2], sd = sigma[i])))
    }
  }
}
ResultadosLS1 <- label.switching::label.switching(z = z, K = 2, p = p, method = "ECR-ITERATIVE-2")
```

```
clasif1 <- Resultados$mean$z
clasif1 <- as.numeric(round(clasif1, 0)) # Clasificacion propuesta por nuestro metodo
clasif2 <- as.numeric(ResultadosLS$clusters)   # Clasificacion una vez realizado el label.switching
Comparacion <- as.data.frame(cbind(clasif1, clasif2))
Comparacion$DIF <- Comparacion$clasif1 - Comparacion$clasif2
sum(Comparacion$DIF)
```

```
## [1] 0
```

Comparando la clasificación previa y la realizada tras solventar el *label.switching* podemos apreciar que no existían diferencias en la clasificación propuesta y que, aunque en este caso no era necesario, el *label switching* funciona correctamente.

ANÁLISIS MULTIVARIANTE

Extrapolamos la construcción sencilla realizada para el caso univariante al caso multivariante. En este caso ya no dispondremos de un criterio visual para determinar a que grupo pertenece una determinada observación. Igualmente, plantearemos una clusterización en tres grupos atendiendo a la propuesta de Zhao 2021 para este mismo dataset.

```
y <- datos
apply(datos, 2, mean)
```

```
##       PCA.PC1       PCA.PC2       PCA.PC3       PCA.PC4       PCA.PC5
## -1.254175e-15  2.607828e-16 -1.039608e-15  6.194410e-16  2.797142e-16
##       PCA.PC6       PCA.PC7
## -3.400761e-16  1.091011e-16
```

Hemos observado que la media por columnas de nuestra matriz de componentes principales es cero. ¿Tiene alguna relevancia esto en nuestro estudio?

```
Modelo_Ejercicio2 <- function()
{
  for(i in 1:293)
  {
    # y[i, 1:7] recoge el valor para cada uno de las 293 localizaciones y 7 CP
    # Asumiremos una distribucion normal multivariante
    # mu[i, ] es el vector de medias para cada localizacion i
    # El valor de mu[i, ] vendrá determinado por mus[, 1], mus[, 2] o mus[, 3] en funcion del cluster
    # Seguimos trabajando como en el caso univariante para la distribucion de z
    y[i, 1:7] ~ dmnorm(mu[i, ], tau[, , z[i]])
    mu[i, 1:7] <- mus[, z[i]]
    z[i] ~ dcat(omega[])
  }
  for (h in 1:3)
  {
    # La distribución Wishart es idonea para estimadores de varianza, como en este caso tau
    # Sus parametros recogen la matriz de varianzas y los grados de libertad
    # En un principio era suficiente con indicar 7 grados de libertad
    mus[1:7, h] ~ dmnorm(rep(0, 7), tau[, , h])
    tau[1:7, 1:7, h] ~ dwish(a, 8)
  }
  omega ~ ddirich(c(1, 1, 1))
}
Datos_Ejercicio2 = list(y = y, a = diag(1, 7))
```

```
set.seed(1)
Resul_Ejercicio2 <- jags(data = Datos_Ejercicio2,
                         inits = Iniciales_Ejercicio2,
                         parameters.to.save = Parametros_Ejercicio2,
                         model.file = Modelo_Ejercicio2,
                         working.directory = NULL, n.iter = 10000, n.burnin = 1000, n.thin = 10)
```

```r
Resultados2 <- Resul_Ejercicio2$BUGSoutput
head(Resultados2$summary)
```

```
##                  mean         sd          2.5%          25%         50%
## deviance 9296.2479552 153.825392 9081.74525984 9117.5215972 9356.156088
## mu[1,1]     0.9305187   6.491925   -8.69077351   -7.7377840    5.036869
## mu[2,1]     4.6503057   2.230602    0.01643349    2.5929851    5.444324
## mu[3,1]     5.5459189   1.833805    0.52763808    5.0468795    5.992147
## mu[4,1]     4.1446253   4.793509   -8.80580250    4.8728629    5.774173
## mu[5,1]     2.5095283   2.634532   -0.73702222    0.6764751    1.500279
##                  75%       97.5%     Rhat n.eff
## deviance 9401.486577 9454.282062 3.682837     4
## mu[1,1]     5.918551    6.948131 5.655055     3
## mu[2,1]     6.300720    7.379411 1.342846    11
## mu[3,1]     6.731072    7.751152 1.386786     9
## mu[4,1]     6.552075    7.609465 1.219660    28
## mu[5,1]     4.597435    7.623443 1.902516     5
```

```r
summary((Resultados2$summary[, 8])) # Informacion R-hat
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   1.139   1.653   2.895   2.512  76.508
```

```r
summary((Resultados2$summary[, 9])) # Informacion simulaciones efectivas
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     1.0     4.0     7.0   132.4    34.0  2700.0
```

Tenemos problemas en este modelo con el *R-hat* y con el número efectivo de simulaciones. ¿Cómo podemos solucionar esto? En el fichero *Modelo2.rda* que utilizamos aquí hemos realizado 10.000 iteraciones. Planteamos también 100.000 iteraciones (*Modelo Extra.rda*) pero la mejora fue prácticamente inexistente, uniendo a ello un tiempo de compilación de casi una hora (que se alarga a aproximadamente 8 horas con el bucle que plantearíamos a continuación).

Obviamos temporalmente este problema para poder proseguir en la valoración del modelo construido. Abordamos, como previamente, el problema de *label.switching*

```r
m <- Resultados2$n.sims
n <- nrow(datos)
K <- 3
z <- Resultados2$sims.list$z
weights <- Resultados2$sims.list$omega
zheta <- Resultados2$sims.list$mus
sigma <- Resultados2$sims.list$tau

p <- array(dim = c(m, n, K))
for(i in 1:m){
  print(i)
  for(j in 1:n){
    for(k in 1:K){
      p[i, j, k] <- (weights[i, k]*dmvnorm(y[j, ], mean = zheta[i, , k], sigma = sigma[i, , , k])) /
                    ((weights[i, 1]*dmvnorm(y[j, ], mean = zheta[i, , 1], sigma = sigma[i, , , 1])) +
                     (weights[i, 2]*dmvnorm(y[j, ], mean = zheta[i, , 2], sigma = sigma[i, , , 2])) +
                     (weights[i, 3]*dmvnorm(y[j, ], mean = zheta[i, , 3], sigma = sigma[i, , , 3])))
    }
  }
}
ResultadosLS2 <- label.switching::label.switching(z = z, K = 3, p = p, method = "ECR-ITERATIVE-2")
```

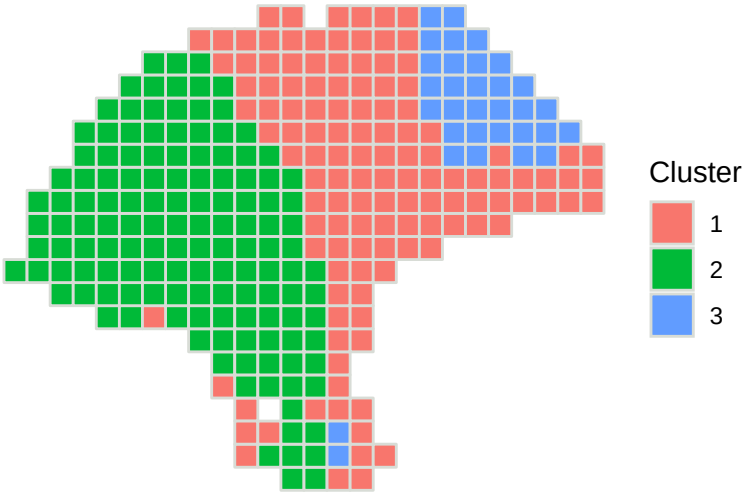
```r
clasif1 <- Resultados2$mean$z
clasif1 <- as.numeric(round(clasif1, 0)) # Clasificacion propuesta por nuestro metodo
clasif2 <- as.numeric(ResultadosLS2$clusters)   # Clasificacion una vez realizado el label.switching
Comparacion <- as.data.frame(cbind(clasif1, clasif2))
Comparacion$DIF <- Comparacion$clasif1 - Comparacion$clasif2
sum(Comparacion$DIF)
```

```
## [1] -17
```

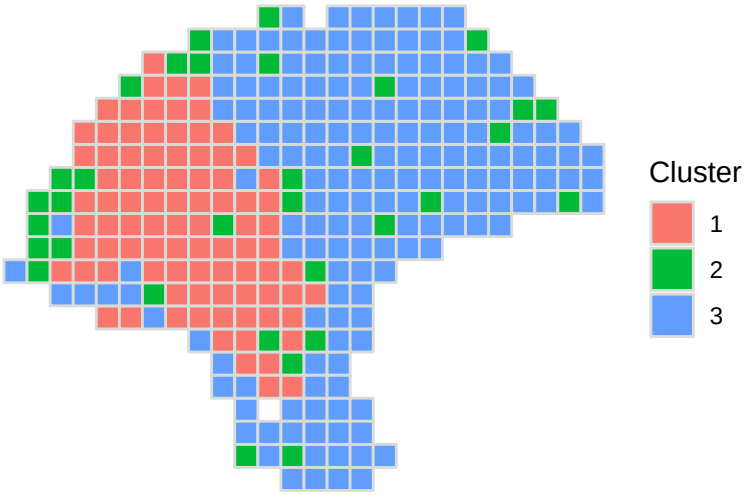
En este caso si que existen diferencias que apuntarían a la necesidad del *label.swiching*. El tiempo de ejecución del bucle para un total de 2700 iteraciones es de aproximadamente 45 minutos. Guardamos los resultados en *ResultadosLS2.rda*. Aprovechamos y guardamos igualmente el primer caso, *ResultadosLS1.rda*.

Llegados a este punto, la clave consistiría en comparar la clasificación propuesta por nuestro modelo con la clasificación que se propone en Zhao 2021 así como con otras técnicas que se mencionan en este mismo artículo.

```r
load("ClasificacionMelanomaZhao.rda")
par(mfrow = c(1, 2))
clusterPlot(ClasificacionMelanomaZhao)
```




```
ClasificacionMelanomaZhao$spatial.cluster <- as.numeric(ResultadosLS2$clusters)
clusterPlot(ClasificacionMelanomaZhao)
```



Vemos que existen diferencias significativas entre las dos clasificaciones propuestas. Este debe ser el punto de partida de nuestras nuevas mejoras y en particular de la aplicación de la estadística espacial.

CONCLUSIONES

Dos de los aspectos últimos que hemos trabajado se han centrado en conseguir la matriz original de recuentos para cada gen y para cada spot y plantear el último modelo aquí descrito a través de otra alternativa como podría ser Nimble.

- **Matriz Original**

En *BayesSpace (Luis).Rmd* hemos realizado un código para obtener la matriz original de recuentos que no venía proporcionada en el dataset original.

```
load("MelanomaZhao.rda")
```

Esto nos podría permitir intentar construir nuestro modelo final con este dataset (valorando por ejemplo lo expuesto en Zhao 2022 para el excesivo número de recuentors iguales a 0) o tratar de desarrollar un análises de componentes principales propio y no utilizar el directamente proporcionado por el artículo. Así, igualmente, conoceríamos también la cantidad de componentes principales necesarias para explicar una determinada cantidad de información o saber cuales de estos podrían presentar una mayor variabilidad.

No obstante, la realización del ACP no ha sido muy satisfactoria. El ampliamente mayor número de variables que de observaciones impide la utilización de la función *princomp()*. Igualmente, la función *principal()* de la librería *psych* tampoco parece funcionar (sin resultados después de media hora compilando tras solicitar solamente una componente).

```
ACP <- princomp(datos)
ACP <- psych::principal(datos, nfactors = 1)
```

Hemos encontrado una alternativa en la función *nipals()* del paquete *chenometrics*. No muestra la proporción de cantidad explicada pero sí devuelve mediante una serie de iteraciones la PCA que buscamos así como el peso de los distintos genes. Replicamos nuestro modelo final con esta nueva PCA. Habría que valorar sí merece la pena hacer hincapié en esta idea.

```
ACP <- nipals(datos, a = 7)
y <- as.data.frame(ACP$T)
```

```r
Modelo_Ejercicio3 <- function()
{
  for(i in 1:293)
  {
    # y[i, 1:7] recoge el valor para cada uno de las 293 localizaciones y 7 CP
    # Asumiremos una distribucion normal multivariante
    # mu[i, ] es el vector de medias para cada localizacion i
    # El valor de mu[i, ] vendrá determinado por mus[, 1], mus[, 2] o mus[, 3] en funcion del cluster
    # Seguimos trabajando como en el caso univariante para la distribucion de z
    y[i, 1:7] ~ dmnorm(mu[i, ], tau[, , z[i]])
    mu[i, 1:7] <- mus[, z[i]]
    z[i] ~ dcat(omega[])
  }
  for (h in 1:3)
  {
    # La distribución Wishart es idonea para estimadores de varianza, como en este caso tau
    # Sus parametros recogen la matriz de varianzas y los grados de libertad
    # En un principio era suficiente con indicar 7 grados de libertad
    mus[1:7, h] ~ dmnorm(rep(0, 7), tau[, , h])
    tau[1:7, 1:7, h] ~ dwish(a, 8)
  }
  omega ~ ddirich(c(1, 1, 1))
}
Datos_Ejercicio3 = list(y = y, a = diag(1, 7))
```

```r
set.seed(1)
Resul_Ejercicio3 <- jags(data = Datos_Ejercicio3,
                         inits = Iniciales_Ejercicio3,
                         parameters.to.save = Parametros_Ejercicio3,
                         model.file = Modelo_Ejercicio3,
                         working.directory = NULL, n.iter = 10000, n.burnin = 1000, n.thin = 10)
```

```r
Resultados3 <- Resul_Ejercicio3$BUGSoutput
head(Resultados3$summary)
```

```
##                mean        sd       2.5%        25%        50%        75%
## deviance 18359.2018 113.54797 18314.3927 18336.3460 18349.4203 18362.6763
## mu[1,1]   -116.3250  43.09682  -161.4361  -141.3387  -129.8727  -115.4158
## mu[2,1]    133.0310   6.10620   124.9355   130.4902   133.1272   135.9420
## mu[3,1]    132.1741  16.12141   124.7073   130.4715   133.0671   135.9176
## mu[4,1]    125.0733  45.13820  -111.6170   130.2367   132.8901   135.8211
## mu[5,1]    132.5938  11.39178   124.8412   130.4797   133.1026   135.9390
##                 97.5%     Rhat n.eff
## deviance 18534.904639 1.004982   450
## mu[1,1]     -5.004595 1.001507  1900
## mu[2,1]    141.579122 1.000990  2700
## mu[3,1]    141.538600 1.036416  2700
## mu[4,1]    141.488876 1.021555   760
## mu[5,1]    141.538600 1.034095  2200
```

```r
summary((Resultados3$summary[, 8])) # Informacion R-hat
```

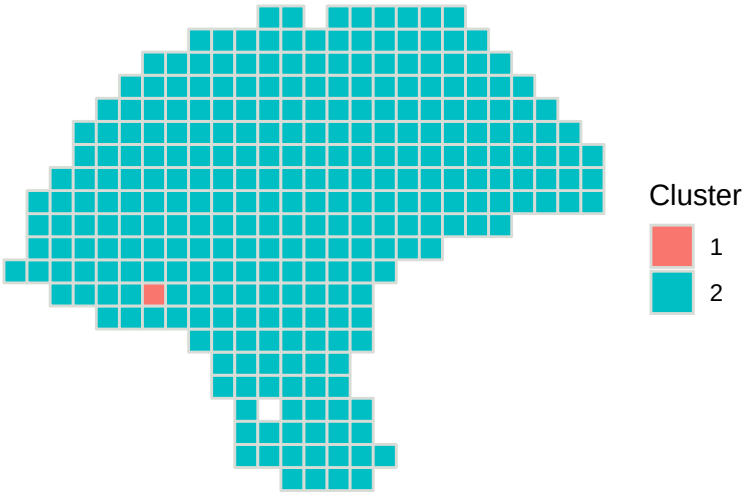
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   1.004   1.024   4.078   1.113 106.141
```

```
summary((Resultados3$summary[, 9])) # Informacion simulaciones efectivas
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     1.0    38.0   260.0   774.5  1225.0  2700.0
```

Merece la pena comentar que aunque todavía existen algunas problemas con el *Rhat* y el número de simulaciones efectivas, estos son mucho menores que los obtenidos con la PCA proporcionada originalmente. No obstante, la clasificación obtenida (sin pasar el label.swiching) es deplorable.

```
load("ClasificacionMelanomaZhao.rda")
Clasificacion <- round(Resultados3$mean$z, 0)
ClasificacionMelanomaZhao$spatial.cluster <- as.numeric(Clasificacion)
clusterPlot(ClasificacionMelanomaZhao)
```



- **Utilización de Nimble**

```
Modelo_EjercicioN <- nimbleCode(
{
  for(i in 1:293)
  {
    # y[i, 1:7] recoge el valor para cada uno de las 293 localizaciones y 7 CP
    # Asumiremos una distribucion normal multivariante
    # mu[i, ] es el vector de medias para cada localizacion i
    # El valor de mu[i, ] vendrá determinado por mus[, 1], mus[, 2] o mus[, 3] en funcion del cluster
    # Seguimos trabajando como en el caso univariante para la distribucion de z
    y[i, 1:7] ~ dmnorm(mu[i], tau[1:7, 1:7, z[i]])
    mu[i] <- mus[1:7, z[i]]
    z[i] ~ dcat(omega[1:3])
  }
  for (h in 1:3)
  {
    # La distribución Wishart es idonea para estimadores de varianza, como en este caso tau
    # Sus parametros recogen la matriz de varianzas y los grados de libertad
    # En un principio era suficiente con indicar 7 grados de libertad
    mus[1:7, h] ~ dmnorm(b, tau[1:7, 1:7, h])
    tau[1:7, 1:7, h] ~ dwish(a, 8)
  }
  omega[1:3] ~ ddirich(1)
}
)

Datos_EjercicioN = list(y = y, a = 1)

Iniciales_EjercicioN <- function()
{
  list(prec = 1)
}

Parametros_EjercicioN <- c("mu", "omega", "mus", "z", "tau")
```

```
set.seed(1)
Resul_EjercicioN <- nimbleMCMC(Modelo_EjercicioN, data = list(y = y, a = 1, b = 0),
                               inits = Iniciales_EjercicioN,
                               monitors = Parametros_EjercicioN,
                               thin = 10, niter = 20000, nburnin = 4000, nchains = 1,
                               summary = TRUE, WAIC = TRUE)
```

El paso de *jags* a *nimble* nos ha generado bastantes problemas de nomenclatura. Hemos ido solucionando muchos de ellos, y aunque la definición del modelo actual es correcta según *nimbleMCMC*, luego genera problemas a la hora de la construcción.

Seguimos igualmente estos días tratando de solucionar el problema y una vez obtenido el modelo tratar de aplicar la componente espacial necesaria para mejorar los resultados.

BIBLIOGRAFÍA

Modelos de Mixturas en Frecuentista

Modelos de Mixturas en Bayesiano

Finite Mixture Models

Distribucion Dirichlet

Distribucion Normal Multivariante

Label Switching, Stephens (2000)

Label Switching, Papastamoulis (2015)

Analisis Componentes Principales

Manual Nimble